

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Румянцев И.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Базовая часть:

1. Реализовать алгоритм Ахо-Корасик для решения задачи точного поиска набора образцов в тексте.

Вход: Текст T ($1 \leq |T| \leq 100000$), число n ($1 \leq n \leq 3000$) и n строк-шаблонов $P = \{p_1, \dots, p_n\}$ ($1 \leq |p_i| \leq 75$). Алфавит: $\{A, C, G, T, N\}$.

Выход: Отсортированный по возрастанию список пар (i, p) , где i – позиция начала вхождения (с 1), p – номер шаблона (с 1).

2. Используя реализацию точного множественного поиска, решить задачу точного поиска для одного образца с джокером.

Вход: Текст T , шаблон P ($1 \leq |P| \leq 40$), символ джокера.

Выход: Отсортированный список позиций вхождений шаблона.

Индивидуальное задание (Вариант 1):

На месте джокера может быть любой символ, **за исключением заданного**. То есть джокер совпадает с любым символом алфавита, кроме одного конкретного (запрещённого).

Требование к реализации: В автомате должны использоваться и суффиксные, и конечные (терминальные, сжатые) ссылки. Код должен поддерживать режим отладочного вывода (построение бора, автомата, описание вершин, процесс поиска).

Описание алгоритма

Базовый алгоритм Ахо-Корасик

Алгоритм позволяет найти все вхождения множества строк-образцов в текст за время, линейно зависящее от длины текста и суммы длин образцов. Он состоит из трёх этапов:

1. Построение бора (префиксного дерева):

Все шаблоны добавляются в дерево. Каждая вершина хранит:

- `children`: хэш-таблицу (словарь Python) переходов по символам.
- `pattern_indices`: список индексов шаблонов, заканчивающихся в этой вершине.
- `suffix_link`: ссылку на наибольший собственный суффикс, который также является префиксом какого-либо шаблона.
- `terminal_link`: ссылку на ближайшую по цепочке суффиксных ссылок вершину, которая является концом какого-либо шаблона.

2. Построение автомата (BFS):

С помощью обхода в ширину от корня вычисляются ссылки:

- Для дочерних вершин корня `suffix_link = 0`.
- Для остальных вершин `suffix_link` ищется переходом по `suffix_link` родителя, пока не найдется вершина, имеющая переход по текущему символу, или не будет достигнут корень.
- **Конечная ссылка (`terminal_link`)** вычисляется так: если вершина, на которую указывает `suffix_link`, сама является концом шаблона, то `terminal_link` указывает на неё. Иначе `terminal_link` копируется из `suffix_link`. Это

ключевая оптимизация, позволяющая находить все совпадения за $O(1)$ на шаг, без полного подъема по суффиксным ссылкам.

3. Поиск в тексте:

Текст обрабатывается посимвольно. При отсутствии прямого перехода алгоритм «откатывается» по `suffix_link`. При достижении новой вершины собираются все совпавшие шаблоны: сначала из `pattern_indices` текущей вершины, затем по цепочке `terminal_link`.

2.2. Модификация для Варианта 1 (Джокер с исключением)

Поскольку классический Ахо-Корасик не поддерживает джокеры напрямую, применяется следующий подход:

1. Шаблон разбивается символом-джокером на массив сплошных подстрок (частей). Запоминаются исходные позиции начала каждой части в шаблоне.

2. Для найденных частей строится стандартный автомат Ахо-Корасик.

3. Автомат ищет все вхождения этих частей в текст.

4. Вводится массив счётчиков `occurrences`. Если часть шаблона найдена, мы вычисляем её потенциальную стартовую позицию в тексте и увеличиваем счётчик для этой позиции.

5. Позиция в тексте считается *кандидатом* на полное совпадение, если счётчик равен количеству частей шаблона (то есть все части нашлись на своих местах).

6. **Финальная проверка (Вариант 1):** Для каждого кандидата проверяются символы текста на позициях джокеров. Если хотя бы один из них равен *запрещённому* символу,

вхождение отбраковывается. Иначе позиция добавляется в ответ.

2.3. Оценка сложности

- **Временная сложность:**

- Построение бора: $O(M)$, где M – суммарная длина шаблонов.
- Построение ссылок (BFS): $O(M)$ амортизированно, так как каждая суффиксная ссылка либо увеличивает глубину, либо уменьшается, но не более чем на 1 за шаг.
- Поиск в тексте: $O(|T| + K)$, где $|T|$ – длина текста, K – общее число найденных вхождений.
- Финальная проверка для Варианта 1: $O(|T| \cdot W)$, где W – количество джокеров в шаблоне ($W \leq 40$).
- **Итоговая временная сложность:** $O(M + |T| + K + |T| \cdot W)$.

- **Пространственная сложность:**

- Хранение бора: $O(M)$ вершин. Использование хэш-таблицы (dict в Python) для хранения переходов children обеспечивает среднее время доступа $O(1)$ и расход памяти, пропорциональный только реально существующим переходам, а не всему алфавиту $|\Sigma|$. Это эффективнее массивов фиксированного размера для разреженных деревьев.
- Массив счётчиков и результатов: $O(|T|)$.
- **Итоговая пространственная сложность:** $O(M + |T|)$.

Тестирование:

Задача 1: Ахо-Корасик (базовый поиск)

Входные данные (Т, n, шаблоны)	Выходные данные (i, p)	Комментарий
NTAG; 3; TAGT, TAG, T	2 2; 2 3	Базовый тест из Stepik
ACGTAACGT; 3; ACG, GTA, CGT	1 1; 2 2; 3 3; 6 1; 7 3	Пересекающиеся вхождения
AAAA; 1; A	1 1; 2 1; 3 1; 4 1	Шаблон из одного символа
AC; 1; ACGT	(пусто)	Шаблон длиннее текста
AGC; 1; TAT	(пусто)	Шаблон не найден в тексте
AAA; 2; A, A	1 1; 1 2; 2 1; 2 2; 3 1; 3 2	Дубликаты шаблонов

Ахо-Корасик с джокером (Вариант 1 — джокер с исключением)

Входные данные (Т, Р, джокер, запрещ.)	Выходные данные (позиции)	Комментарий
ACTANCA; A\$\$A\$; \$; \$	1	Базовый тест из Stepik
ACGAG; A\$A; \$; G	(пусто)	На месте джокера запрещённый символ
ACGAG; A\$A; \$; T	1	На месте джокера допустимый символ
AAAAA; A\$A; \$; X	1; 2; 3	Пересекающиеся вхождения
AAA; \$\$\$; \$; X	1	Шаблон только из

		джокеров
AC; A\$A\$; \$; \$	(пусто)	Шаблон длиннее текста
ACGTACGT; G\$G; \$; A	1; 5	Джокер не может быть A

Вывод

В ходе выполнения лабораторной работы был реализован алгоритм Ахо-Корасик для множественного поиска образцов в тексте. Особое внимание было уделено выполнению строгих требований: в автомате реализованы и активно используются как суффиксные, так и **конечные (терминальные) ссылки**, что гарантирует оптимальную оценку временной сложности $O(|T| + K)$ на этапе поиска и предотвращает деградацию производительности.

Для хранения переходов в вершинах бора использована хэш-таблица (dict), что обеспечивает среднее время доступа $O(1)$ и экономит память по сравнению с массивами фиксированного размера, сохраняя при этом высокую скорость работы.

В рамках индивидуального задания (**Вариант 1**) алгоритм был успешно модифицирован для поддержки джокера с исключением. Реализован подход с разбиением шаблона на сплошные подстроки, их поиском классическим автоматом и последующей валидацией позиций джокеров на предмет отсутствия запрещённого символа. Разработан механизм отладочного вывода, позволяющий наглядно продемонстрировать преподавателю все этапы работы алгоритма: от построения бора до пошагового использования автомата.

